Tejaswi Khode BEB221

Implement N-Gram model for the given text input.

Theory

An **N-Gram Model** is a statistical language model used in **Natural Language Processing (NLP)** to predict the next word in a sequence based on the previous word(s). An **N-gram** is a sequence of **N consecutive words** extracted from a text.

- **Unigram (N = 1):** Single words (e.g., *machine*, *learning*)
- **Bigram (N = 2):** Two consecutive words (e.g., *machine learning*)
- **Trigram (N = 3):** Three consecutive words (e.g., *machine learning model*)

The N-Gram model works by counting the frequency of word sequences in a dataset and calculating their probabilities. In this experiment, the **Resume Dataset** is preprocessed by converting text to lowercase, removing special characters, and tokenizing the text. Unigrams, bigrams, and trigrams are then generated to identify common word patterns, predict the next word, and analyze language structure.

N-Gram models are widely used in **text prediction, autocomplete, spell checking, machine translation, speech recognition, and language modeling**. They are simple, efficient, and serve as the foundation for many NLP applications.

In [23]:
```
import re
import pandas as pd
from collections import Counter, defaultdict
import nltk
nltk.download('punkt')
from nltk.tokenize import word_tokenize
from nltk.util import ngrams
print("Setup complete.")
```

```
Setup complete.
[nltk_data] Downloading package punkt to /home/computer/nltk_data...
[nltk_data]   Package punkt is already up-to-date!
```

In [24]:
```
df = pd.read_csv("UpdatedResumeDataSet.csv")
print("Shape :", df.shape)
df[['Category','Resume']].head()
```

```
Shape : (962, 2)
```

Out[24]:

	Category	Resume
0	Data Science	Skills * Programming Languages: Python (pandas...
1	Data Science	Education Details \r\nMay 2013 to May 2017 B.E...
2	Data Science	Areas of Interest Deep Learning, Control Syste...
3	Data Science	Skills â¯¢ R â¯¢ Python â¯¢ SAP HANA â¯¢ Table...
4	Data Science	Education Details \r\n MCA YMCAUST, Faridab...

In [25]:
```
def generate_ngrams(text, n):
    if not isinstance(text, str) or text.strip()=="":
        return []
    text=text.lower()
    text=re.sub(r'http\S+',' ',text)
    text=re.sub(r'[^a-z\s]',' ',text)
    text=re.sub(r'\s+',' ',text)
    tokens=word_tokenize(text)
    n_grams=list(ngrams(tokens,n))
    return n_grams
```

In [75]:
```
sample_resume=df['Resume'].iloc[0]
print("Original Resume:\n")
print(sample_resume[:200])
unigrams=generate_ngrams(sample_resume,1)
bigrams=generate_ngrams(sample_resume,2)
trigrams=generate_ngrams(sample_resume,3)
print("\nFirst 10 Unigrams\n")
print(unigrams[:10])
print("\nFirst 10 Bigrams\n")
print(bigrams[:10])
print("\nFirst 10 Trigrams\n")
print(trigrams[:10])
```

```
Original Resume:

Skills * Programming Languages: Python (pandas, numpy, scipy, scikit-learn, matplotlib), Sql, Java, JavaScript/JQuery. * Machine learning: Regression, SVM, NaÃ¯ve Bayes, KNN, Random Forest, Decision T

First 10 Unigrams

[('skills',), ('programming',), ('languages',), ('python',), ('pandas',), ('numpy',), ('scipy',), ('scikit',), ('learn',), ('matplotlib',)]

First 10 Bigrams

[('skills', 'programming'), ('programming', 'languages'), ('languages', 'python'), ('python', 'pandas'), ('pandas', 'numpy'), ('numpy', 'scipy'), ('scipy', 'scikit'), ('scikit', 'learn'), ('learn', 'matplotlib'), ('matplotlib', 'sql')]

First 10 Trigrams

[('skills', 'programming', 'languages'), ('programming', 'languages', 'python'), ('languages', 'python', 'pandas'), ('python', 'pandas', 'numpy'), ('pandas', 'numpy', 'scipy'), ('numpy', 'scipy', 'scikit'), ('scipy', 'scikit', 'learn'), ('scikit', 'learn', 'matplotlib'), ('learn', 'matplotlib', 'sql'), ('matplotlib', 'sql', 'java')]
```

In [58]:
```
def ngram_frequency(text,n):
    grams=generate_ngrams(text,n)
    return Counter(grams)
```

In [59]:
```
bigram_freq=ngram_frequency(sample_resume,2)
print("Top 10 Bigrams\n")
for gram,count in bigram_freq.most_common(10):
    print(gram,"->",count)
```

```
Top 10 Bigrams

('topic', 'modelling') -> 5
('tools', 'technologies') -> 4
('scikit', 'learn') -> 3
('sentiment', 'analysis') -> 3
('data', 'science') -> 3
('analytics', 'and') -> 3
('of', 'the') -> 3
('technologies', 'python') -> 3
('analysis', 'on') -> 3
('elastic', 'search') -> 3
```

In [60]:
```
def build_bigram_model(corpus):
    bigram_counts=defaultdict(Counter)
    unigram_counts=Counter()
    for text in corpus:
        if not isinstance(text,str):
            continue
        text=text.lower()
        text=re.sub(r'[^a-z\s]',' ',text)
        tokens=word_tokenize(text)
        unigram_counts.update(tokens)
        for w1,w2 in ngrams(tokens,2):
            bigram_counts[w1][w2]+=1
    return bigram_counts,unigram_counts
```

In [61]:
```
corpus=df["Resume"].tolist()
bigram_counts,unigram_counts=build_bigram_model(corpus)
print("Vocabulary Size :",len(unigram_counts))
```

```
Vocabulary Size : 7279
```

In [62]:
```
def bigram_probability(w1,w2,bigram_counts,unigram_counts):
    if unigram_counts[w1]==0:
        return 0
    return bigram_counts[w1][w2]/unigram_counts[w1]
```

In [63]:
```
print("P('learning'|'machine') =",
      round(bigram_probability('machine','learning',
                               bigram_counts,
                               unigram_counts),4))
print("P('developer'|'software') =",
      round(bigram_probability('software','developer',
                               bigram_counts,
                               unigram_counts),4))
```

```
P('learning'|'machine') = 0.5683
P('developer'|'software') = 0.1037
```

In [64]:
```
def predict_next_word(word,bigram_counts,top_n=5):
    word=word.lower()
    if word not in bigram_counts:
        return []
    return bigram_counts[word].most_common(top_n)
```

In [65]:
```
for word in ["machine","data","python","software"]:
    print("\nNext words after",word)
    print(predict_next_word(word,bigram_counts))
```

```
Next words after machine
[('learning', 179), ('in', 16), ('technical', 10), ('svm', 8), ('and', 8)]

Next words after data
[('science', 174), ('from', 156), ('warehouse', 98), ('into', 96), ('to', 86)]

Next words after python
[('exprience', 116), ('developer', 88), ('and', 20), ('python', 20), ('selenium', 16)]

Next words after software
[('developer', 92), ('testing', 76), ('development', 74), ('description', 46), ('and', 37)]
```

In [66]:
```
def extract_ngrams(text,n):
    grams=generate_ngrams(text,n)
    return [' '.join(g) for g in grams]
```

In [67]:
```
df["Bigrams"]=df["Resume"].apply(lambda x:extract_ngrams(x,2))
df["Trigrams"]=df["Resume"].apply(lambda x:extract_ngrams(x,3))
df[['Category','Bigrams','Trigrams']].head()
```

Out[67]:

	Category	Bigrams	Trigrams
0	Data Science	[skills programming, programming languages, la...	[skills programming languages, programming lan...
1	Data Science	[education details, details may, may to, to ma...	[education details may, details may to, may to...
2	Data Science	[areas of, of interest, interest deep, deep le...	[areas of interest, of interest deep, interest...
3	Data Science	[skills r, r python, python sap, sap hana, han...	[skills r python, r python sap, python sap han...
4	Data Science	[education details, details mca, mca ymcaust, ...	[education details mca, details mca ymcaust, m...

In [73]:
```
all_bigrams=Counter([bg for row in df["Bigrams"] for bg in row])
print("Top 10 Bigrams\n")
for gram,count in all_bigrams.most_common(10):
    print(gram,"->",count)
```

```
Top 10 Bigrams

exprience months -> 2170
details company -> 1194
exprience less -> 1137
less than -> 1137
than year -> 1137
of the -> 1064
skill details -> 962
year months -> 919
pvt ltd -> 873
education details -> 820
```

In [74]:
```
all_trigrams=Counter([tg for row in df["Trigrams"] for tg in row])
print("Top 10 Trigrams\n")
for gram,count in all_trigrams.most_common(10):
    print(gram,"->",count)
```

```
Top 10 Trigrams

exprience less than -> 1137
less than year -> 1137
than year months -> 913
monthscompany details company -> 746
exprience monthscompany details -> 522
pvt ltd description -> 486
education details january -> 330
as well as -> 228
than year monthscompany -> 224
year monthscompany details -> 224
```

In [70]:
```
def generate_sentence(start_word,length=12):
    sentence=[start_word]
    current=start_word
    for i in range(length):
        nxt=predict_next_word(current,bigram_counts,1)
        if len(nxt)==0:
            break
        current=nxt[0][0]
        sentence.append(current)
    return " ".join(sentence)
```

In [71]:
```
print(generate_sentence("machine"))
print(generate_sentence("data"))
print(generate_sentence("software"))
```

```
machine learning exprience months python exprience months python exprience months python exprience months
data science pune maharashtra state board january b e commerce product features for
software developer skill details company details company details company details company details company
```

In [72]:
```
df[['Category','Resume','Bigrams','Trigrams']].to_csv(
    "NGram_Resume_Dataset.csv",
    index=False
)
print("File Saved Successfully.")
```

```
File Saved Successfully.
```